

Advanced computerarchitecture
Exercise 5 Report

(B-KUL-JPI30R)

Wout Van Steenbergen

March 24, 2024



Session Date: March 18, 2024

Instructor: Alex Marinšek

1 Introduction

This report investigates the performance differences between synchronous and asynchronous execution in parallel GPU computing, focusing on the reduction operation—specifically sum, product, minimum, and maximum—across four independently generated arrays. During the exercise two functions are run: the first executes array operations in sequence, synchronously. the second generates arrays while processing previous data asynchronously on the GPU. The goal is to compare the total execution times of both approaches, including CPU and GPU computations, to understand the impact of execution strategies on computational efficiency.

2 Synchronous Execution

In synchronous execution, tasks are performed one after the other, each task starting only after the previous one has completed. This sequential execution model is straightforward but can lead to inefficiencies, especially in operations that involve waiting for external processes or devices, like GPUs or I/O operations. All operations (sum, prod, min, max) on the four arrays are executed in sequence Each array is generated one at a time using a for loop, and each operation waits for the previous one to complete before starting. This approach might result in underutilization of the GPU, as it waits idle for the CPU to prepare and send the next batch of data.

3 Asynchronous Execution

Asynchronous execution allows tasks to run concurrently, not waiting for each task to complete before starting the next one. This model is more complex but can significantly increase efficiency by overlapping computation with data preparation and other computations. While one array's data is being processed on the GPU (sum, prod, min, max), the next array is generated simultaneously by the CPU. This approach minimizes idle time, as the GPU and CPU can work in parallel, significantly reducing the overall execution time.

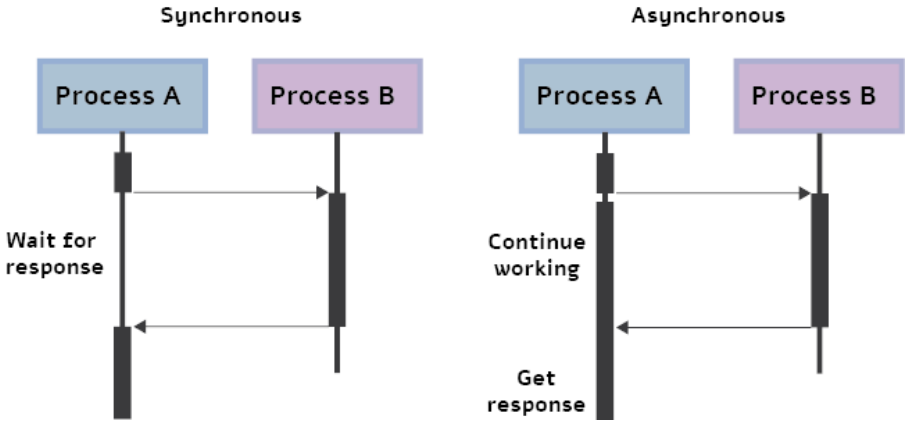


Figure 1: Synchronous - vs Asynchronous execution [1].

4 Results

Based on the graph in Figure 2, it appears that the synchronous (sync) execution has a slightly faster execution time than the asynchronous (async) execution for the task of array reduction with operations such as sum, product, minimum, and maximum. This observation is intriguing, as one would typically expect async execution to be faster due to parallelization of tasks. In order to obtain a more visible result, a fixed number of threads (64) and blocks (1) was chosen on purpose for a large number of elements (10k) so that the generation time is as large as the calculation time. when one now runs the two different executions, there should be a clear difference. The second graph in Figure 3, presents this spread. Yet again, the sync execution exhibits a tighter spread and a lower median time than the async execution.

There could be several reasons why sync execution is outperforming async in this scenario. Asynchronous execution involves managing concurrent operations, which can introduce overhead that doesn't exist in synchronous execution. This overhead may negate the time saved by parallelizing the tasks. The GPU scheduler may be more efficient at handling a batch of operations all at once (sync) rather than interleaving them with array generation (async). It's also important to consider the specific details of the hardware and software environment in which these tests were conducted, as different configurations can significantly affect the performance of sync versus async execution.

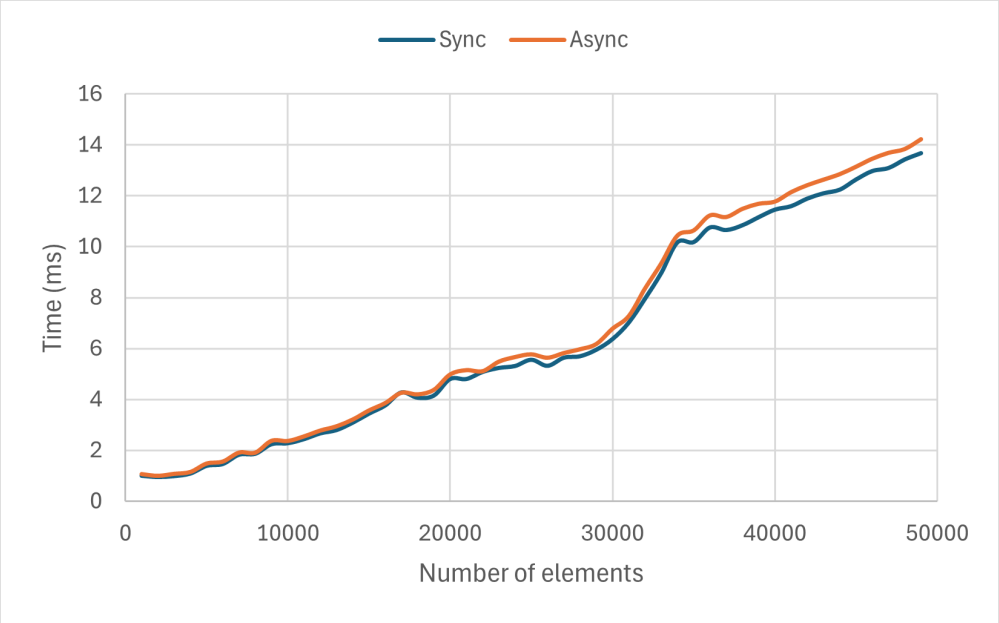


Figure 2: Synchronous - vs Asynchronous execution with increasing number of elements.

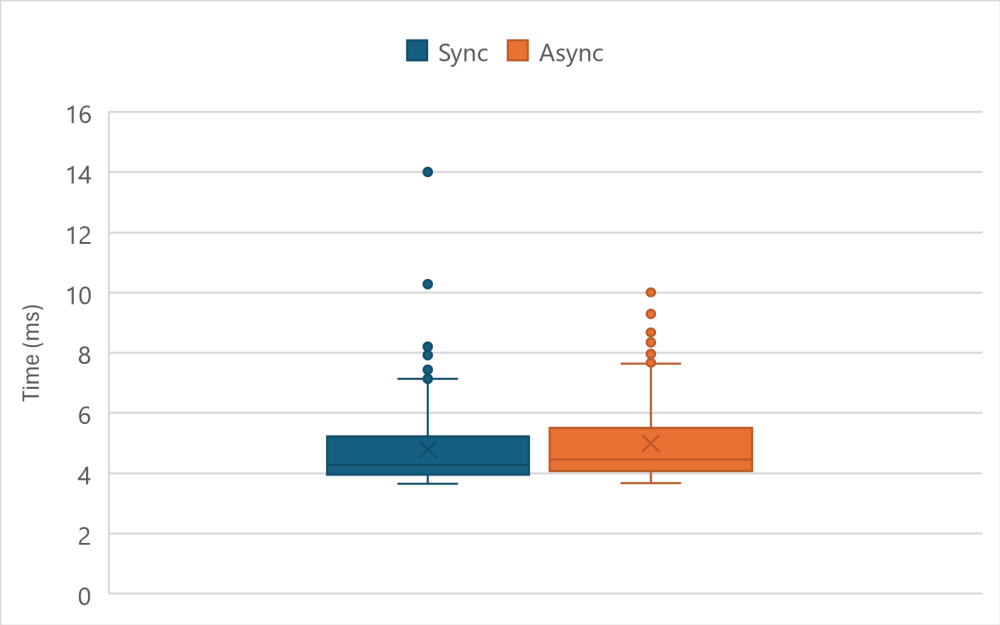


Figure 3: Synchronous - vs Asynchronous execution with same generation and calculation time.

References

[1] A. Geifman and Deci, "The correct way to measure inference time of deep neural networks." https://deci.ai/blog/measure-inference-time-deep-neural-networks/, 2023. Accessed: 24 March 2024.